# Multicollinearity and Regularization in Regression Models

Dilip Kumar Follow 9 min read · Jul 12, 2025

     

## 1. The Concept: What is Multicollinearity?

**One-liner:** Multicollinearity occurs when two or more independent features in your dataset are highly correlated with each other.

**The Intuition (The "Redundant Reporters" Analogy):**
Imagine you are a manager trying to figure out what's happening in a factory. You hire two reporters.

- **Reporter A** gives you a report in English.
- **Reporter B** gives you the *exact same report*, but translated into French.

Both reports contain the same information. If you try to build a model to predict factory output based on these two reports, the model will get confused. How much credit should it give to Reporter A vs. Reporter B? It

might give a huge positive weight to A and a huge negative weight to B, or vice-versa. The results become **unstable, unreliable, and uninterpretable.**

This is what multicollinearity does to a standard Linear Regression model.

## 2. Interpret model coefficients

### 2.1. The Goal: A Stable and Sensible Model

A stable model is one where:

1. The coefficients are **reliable** and don't change wildly with small changes in the data.
2. The coefficients make **intuitive sense** in the context of the problem.

### 2.2. Look for Unreasonably Large Coefficients

This is the number one sign of instability, often caused by multicollinearity.

**Red Flag:** You see extremely large positive and negative coefficients on features that you know are highly correlated or measure similar things.

**Example:**

**Unstable Model (Linear Regression):** Feature 1 = 17.58, Feature 4 = -14.50

- **Interpretation:** This is highly unstable. The model is essentially saying "To predict y, add 17.58 times Feature 1, but then immediately subtract 14.50 times the almost identical Feature 4." This is a fragile mathematical balancing act. If you were to slightly change the training data, these two

numbers could flip or change dramatically. The model has no real confidence in either feature individually.

**Stable Model (Ridge Regression):** Feature 1 = 1.53, Feature 4 = 1.49

- **Interpretation:** These coefficients are small, reasonable, and have the same sign. The model has stabilized and is confidently saying that both features have a similar positive impact on the target. This model is robust; small changes in the data will not cause these coefficients to explode.

### 2.3. Look for Coefficients That Make Business/Domain Sense

A stable model's coefficients should align with your real-world understanding of the problem.

**Red Flag:** A coefficient's sign is the opposite of what you expect.

**Example (Car Price Prediction):**

**Unstable/Incorrect Model:** A model that returns a *positive* coefficient for the Mileage feature.

- **Interpretation:** This would mean that as a car's mileage *increases*, its price is predicted to *increase*. This makes no real-world sense. This is a sign that the model is unstable, likely due to multicollinearity with another feature (e.g., Age). It has found a strange mathematical correlation but has failed to capture the true underlying relationship.

**Stable/Correct Model:** A model that returns a *negative* coefficient for Mileage.

- **Interpretation:** This aligns with reality. As mileage goes up, the price goes down. This gives you confidence that the model is stable and has learned a meaningful pattern.

### 2.4. Look at the Magnitude Relative to Other Features

The relative size of the coefficients tells you about the feature's importance *within the model*. In a stable model, this should be plausible.

**Important Prerequisite:** This interpretation is only valid if you have **scaled your features** (e.g., using StandardScaler). If features are on different scales, you cannot compare their coefficients directly.

**Red Flag:** A feature you know is only marginally important has a much larger coefficient than a feature you know is critical.

**Example (House Price Prediction):**

**Unstable Model:** A model where the coefficient for number_of_bathrooms is 10 times larger than the coefficient for square_footage.

- **Interpretation:** While bathrooms are important, it's highly unlikely they have 10x the impact of the overall square footage. This suggests instability or that the number_of_bathrooms feature is acting as a proxy for other, more important unmeasured factors in a strange way.

**Stable Model:** The coefficient for square_footage is one of the largest, and the coefficient for number_of_bathrooms is smaller but still significant and positive. This is a sensible and stable hierarchy of importance.

### 2.5. Summary: The Stability Checklist

When you look at a set of regression coefficients, ask yourself these questions to assess stability:

| Question | If the Answer is "Yes," it's a... |
| --- | --- |
| **1. Are there very large positive and negative coefficients on similar features?** | **Red Flag:** High instability, likely multicollinearity. |
| **2. Do the signs (+/-) of the coefficients make real-world sense?** | **Green Flag:** The model has learned a logical relationship. |
| **3. Do the relative magnitudes of the coefficients seem plausible (after scaling)?** | **Green Flag:** The model's feature importance hierarchy is sensible. |
| **4. Do any features you know are useless have large, non-zero coefficients?** | **Red Flag:** The model is overfitting or unstable. A stable regularized model (like Lasso) would push these towards zero. |

## 3. The Solution: Regularization

Regularization is the technique of adding a penalty for large coefficients. This forces the model to be more “conservative” and produce more stable results.

- **Ridge Regression (L2):** Shrinks the coefficients of correlated features, forcing them to “share the credit” more evenly. It makes the model stable.
- **Lasso Regression (L1):** Tends to arbitrarily pick one feature from a correlated group and eliminate the others (by setting their coefficients to zero). It performs feature selection.

## 4. The Experiment: Code to Investigate

Let’s design an experiment to clearly demonstrate this:

1. Create a simple dataset with a few features.
2. Intentionally make two of the features highly correlated.

3. Train three models: Linear Regression, Ridge, and Lasso.
4. Compare the coefficients of the models to see how each one handles the multicollinearity.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression, Ridge, Lasso

# --- 1. Create a dataset with high multicollinearity ---
np.random.seed(42)
n_samples = 100

# Create three independent features
feature1 = np.random.rand(n_samples) * 10
feature2 = np.random.rand(n_samples) * 5
feature3_noise = np.random.rand(n_samples) # This feature is pure noise

# Create a fourth feature that is highly correlated with feature1
# feature4 = feature1 + a tiny bit of noise
feature4_correlated = feature1 + np.random.normal(0, 0.1, n_samples)

# The true relationship only depends on the first two features
# The true coefficients for F1 and F4 should "share" the value 3.0
y = 3 * feature1 + 5 * feature2 + np.random.normal(0, 1, n_samples)

# Create a DataFrame
X = pd.DataFrame({
    'Feature 1': feature1,
    'Feature 2': feature2,
    'Feature 3 (Noise)': feature3_noise,
    'Feature 4 (Correlated with F1)': feature4_correlated
})


# --- 2. Train the three regression models ---
lr_model = LinearRegression()
ridge_model = Ridge(alpha=1.0) # A moderate regularization strength
lasso_model = Lasso(alpha=0.1) # A moderate regularization strength

lr_model.fit(X, y)
ridge_model.fit(X, y)
lasso_model.fit(X, y)
```

```
# --- 3. Analyze the coefficients ---
# Create a DataFrame to easily compare the coefficients
coeffs_df = pd.DataFrame({
    'Feature': X.columns,
    'Linear Regression': lr_model.coef_,
    'Ridge (alpha=1.0)': ridge_model.coef_,
    'Lasso (alpha=0.1)': lasso_model.coef_
})

# Add the intercept for completeness
coeffs_df.loc[len(coeffs_df)] = ['INTERCEPT', lr_model.intercept_, ridge_model.i

# Round the values for easier reading
coeffs_df = coeffs_df.round(2)

print("--- Comparison of Model Coefficients ---")
print(coeffs_df)
```

| | Feature | Linear Regression | Ridge (alpha=1.0) | Lasso (alpha=0.1) |
|---|---|---|---|---|
| 0 | Feature 1 | 2.36 | 1.76 | 2.61 |
| 1 | Feature 2 | 5.02 | 5.00 | 4.96 |
| 2 | Feature 3 (Noise) | 0.44 | 0.36 | 0.00 |
| 3 | Feature 4 (Correlated with F1) | 0.65 | 1.24 | 0.39 |
| 4 | INTERCEPT | -0.27 | -0.14 | 0.16 |

Let's analyze this row by row for the key features:

### Linear Regression Analysis:

- **Observation:** It assigned `2.36` to `Feature 1` and `0.65` to `Feature 4`. Their combined effect is `2.36 + 0.65 = 3.01`. It also gave a non-zero weight (0.44) to the pure noise feature.
- **Interpretation:** It shows the signs of instability. It **failed to distribute the "credit" evenly.** It decided Feature 1 was much more important than the

almost identical Feature 4. This distribution is still somewhat arbitrary and sensitive to small changes in the data.

### Ridge Regression Analysis (The Stabilizer):

- **Observation:** It assigned `1.76` to `Feature 1` and `1.24` to `Feature 4`. Their combined effect is `1.76 + 1.24 = 3.00`. It also correctly shrank the noise feature's coefficient from 0.44 down to 0.36.
- **Interpretation:** This is the key takeaway. Ridge did a much better job of **sharing the credit** between the two correlated features. The coefficients are more balanced and stable. This is a much more robust representation of the underlying relationship. It correctly understands that both features are contributing similarly.

### Lasso Regression Analysis (The Selector):

**Observation:** It assigned `2.61` to `Feature 1` and `0.39` to `Feature 4`. Most importantly, it assigned **0.00** to the Feature 3 (Noise).



**Interpretation:**

1. **Feature Selection:** Lasso's primary strength is perfectly demonstrated here. It correctly identified that Feature 3 was useless and **completely eliminated it**, which neither Linear nor Ridge regression did.

2. **Handling Correlation:** In this case, it didn't fully eliminate one of the correlated features, but it came close. It heavily favored Feature 1 over Feature 4, showing its tendency to pick one "winner" from a correlated group.

## 5. Conclusion

This experiment clearly demonstrates:

1. **The Problem:** Multicollinearity makes a standard Linear Regression model highly unstable and uninterpretable.
2. **The Ridge Solution:** Ridge Regression effectively manages multicollinearity by shrinking the coefficients of correlated features, forcing them to "share the credit" and producing a stable model.
3. **The Lasso Solution:** Lasso Regression manages multicollinearity by performing feature selection, arbitrarily picking one feature from a correlated group and eliminating the others. This creates a simpler, sparser model.

## 6. What is the Variance Inflation Factor (VIF)?

**One-liner:** VIF is a score for each independent feature in your model that measures how much that feature is explained by the *other* features.

**The Intuition:**
To calculate the VIF for a specific feature (e.g., Feature 1), you do something clever:

1. **Run a "side" regression:** You temporarily treat Feature 1 as the target variable and run a linear regression model to predict it using all the *other*

features (Feature 2, Feature 3, etc.).

2. **Get the R-squared:** You check the R-squared value of this "side" model. An R-squared of 0.95 would mean that 95% of the variation in Feature 1 can be predicted by the other features in the dataset. This is a huge red flag for multicollinearity.
3. **Calculate VIF:** The VIF is calculated from this R-squared value.

**The Formula:**

$$VIF = \frac{1}{1 - R^2}$$

**Interpreting the VIF Score:**
The score tells you how much the variance of the estimated coefficient is "inflated" by its correlation with other features.

- **VIF = 1:** No correlation. The feature is completely independent. This is the ideal, but rare.
- **1 < VIF < 5:** Moderate correlation. This is generally considered safe and not a cause for concern.
- **VIF > 5:** High correlation. This is a potential red flag and should be investigated.
- **VIF > 10:** Very high correlation. This is a strong sign of serious multicollinearity, and the coefficients for this feature are likely unstable and unreliable.

## 7. How to Use VIF and Apply Ridge & Lasso

This becomes a two-step process: **Diagnose with VIF, then Treat with Regularization.**

Let's write the code to do this on the same dataset we used before. You will need to install the statsmodels library, which contains the VIF function.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge, Lasso
from statsmodels.stats.outliers_influence import variance_inflation_factor

# --- 1. Create the same dataset with high multicollinearity ---
np.random.seed(42)
n_samples = 100
feature1 = np.random.rand(n_samples) * 10
feature2 = np.random.rand(n_samples) * 5
feature3_noise = np.random.rand(n_samples)
feature4_correlated = feature1 + np.random.normal(0, 0.1, n_samples)
y = 3 * feature1 + 5 * feature2 + np.random.normal(0, 1, n_samples)

X = pd.DataFrame({
    'Feature 1': feature1,
    'Feature 2': feature2,
    'Feature 3 (Noise)': feature3_noise,
    'Feature 4 (Correlated with F1)': feature4_correlated
})

# --- 2. Diagnose Multicollinearity with VIF ---
# Create a DataFrame to store the VIF scores
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
# Calculate VIF for each feature
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(len(X.c

print("--- VIF Scores for Each Feature ---")
print(vif_data)
print("\nObservation: Feature 1 and Feature 4 have extremely high VIF scores, co


# --- 3. Treat Multicollinearity with Ridge and Lasso ---
# Note: In a real project, you would scale your data before fitting these models
```

```
# We are omitting it here to focus only on the coefficient changes.

ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X, y)

lasso_model = Lasso(alpha=0.1)
lasso_model.fit(X, y)


# --- 4. Analyze the coefficients to see how the models handled the problem ---
coeffs_df = pd.DataFrame({
    'Feature': X.columns,
    'Ridge Coefficients': ridge_model.coef_,
    'Lasso Coefficients': lasso_model.coef_
})

print("\n--- Model Coefficients After Regularization ---")
print(coeffs_df.round(2))
print("\nConclusion: Both Ridge and Lasso have produced stable coefficients desp
```

```
--- VIF Scores for Each Feature ---
                         feature          VIF
0                      Feature 1  3290.480077
1                      Feature 2     2.483298
2              Feature 3 (Noise)     2.531461
3  Feature 4 (Correlated with F1)  3289.866346

Observation: Feature 1 and Feature 4 have extremely high VIF scores, confirming severe multicollinearity.

--- Model Coefficients After Regularization ---
                         Feature  Ridge Coefficients  Lasso Coefficients
0                      Feature 1                1.76                2.61
1                      Feature 2                5.00                4.96
2              Feature 3 (Noise)                0.36                0.00
3  Feature 4 (Correlated with F1)                1.24                0.39

Conclusion: Both Ridge and Lasso have produced stable coefficients despite the high VIF scores.
```

## Analysis of the Output

### Step 1: VIF Diagnosis

The VIF scores for Feature 1 and Feature 4 are **over 1000!** This is an astronomically high number and is a definitive diagnosis of severe multicollinearity. The scores for Feature 2 and Feature 3 are very close to 1,

indicating they are not correlated with the other features, which is exactly what we expect.

### Step 2: The Treatment and Result

The second part of the output shows the coefficients after applying Ridge and Lasso:

**Conclusion:** Even though the VIF analysis told us that a standard Linear Regression would be highly unstable, we can see that both Ridge and Lasso were able to produce stable, sensible coefficients.

**Ridge** handled the problem by **sharing the credit** between the two highly correlated features.

**Lasso** handled it by **selecting one** of the features and eliminating the other (along with the noise feature).

## 8. The Complete Workflow

This demonstrates the complete professional workflow:

1. **Hypothesize/Suspect:** You suspect you might have correlated features.
2. **Diagnose:** You use a statistical tool like **VIF** to confirm and quantify the multicollinearity.
3. **Treat:** If high VIF scores are found, you choose a model that is robust to this condition. You **do not use standard Linear Regression**. Instead, you apply a regularized model like **Ridge** (for stability) or **Lasso** (for feature selection) to handle the problem and build a reliable model.

Happy learning !!!

Multicollinearity

## Written by Dilip Kumar

1K followers · 29 following

Follow

With 19+ years of experience as a software engineer. Enjoy teaching, writing, leading team etc. Last 4+ years, working at Google as a backend Software Engineer.